Homework 1 — Solutions

CS-UY 2413: Design & Analysis of Algorithms

Fall 2026 • Prof. Lisa Hellerstein

Conventions. Arrays are indexed beginning at 1. The notation $A[i : j]$ denotes the elements in positions $i, i+1, \ldots, j$ of the array A (position j is included); if $i > j$ then no positions are included.

1. Warm-up exercises.

   (a) What is $\log_3 81$?

   Solution. Write 81 as a power of 3, then apply the definition of the logarithm:

   $$81 = 3^4 \qquad \Longrightarrow \qquad \log_3 81 = \log_3 3^4 = 4.$$

   Hence $\log_3 81 = 4$.

   (b) What is 2^4?

   Solution. $2^4 = 2 \cdot 2 \cdot 2 \cdot 2 = 16$.

   (c) What is $4!$?

   Solution. $4! = 4 \times 3 \times 2 \times 1 = 24$.

   (d) What is $\binom{4}{2}$?

   Solution.

   $$\binom{4}{2} = \frac{4!}{(4-2)!\,2!} = \frac{4 \times 3 \times \cancel{2 \times 1}}{(2 \times 1) \cdot \cancel{(2 \times 1)}} = \frac{4 \times 3}{2} = \frac{12}{2} = 6.$$

2. Starting with one dollar and doubling every day afterwards, how many days until you have at least \$265.4 billion?

Solution. After n days you have 2^n dollars. (Checking this against the statement of the problem: after 3 days you have $2^3 = \$8$, as claimed.) We therefore want the smallest integer n satisfying

$$2^n \;\geq\; \$265.4 \text{ billion} \;=\; 2.654 \times 10^{11}.$$

Taking natural logarithms of both sides,

$$n \ln 2 \;\geq\; \ln\bigl(2.654 \times 10^{11}\bigr) \qquad \Longrightarrow \qquad n \;\geq\; \frac{\ln\bigl(2.654 \times 10^{11}\bigr)}{\ln 2} \;=\; 37.949\ldots$$

Since n must be a whole number of days, we round *up* to $n = 38$.

To confirm that the rounding went the right way, we check the two neighbouring days:

$$2^{37} \approx \$137.4 \text{ billion} \;<\; \$265.4 \text{ billion}, \qquad 2^{38} \approx \$274.9 \text{ billion} \;\geq\; \$265.4 \text{ billion}.$$

Day 37 is not enough, and day 38 is the first day that suffices.

Answer: 38 days.

Alternative Solution (taking $\log_2$ instead of $\ln$). Starting again from $2^n \geq 2.654 \times 10^{11}$, we can instead take $\log_2$ of both sides directly, which is arguably more natural here since the base of the exponential is already 2:

$$\log_2(2^n) \;\geq\; \log_2\bigl(2.654 \times 10^{11}\bigr) \qquad \Longrightarrow \qquad n \;\geq\; \log_2\bigl(2.654 \times 10^{11}\bigr) \;=\; 37.949\ldots$$

This is the same inequality as before (the two approaches agree, since $\log_2 x = \ln x / \ln 2$), so rounding up again gives $n = 38$ days.

3. The nine justices of the Supreme Court each shake hands with every other justice exactly once. How many handshakes result?

Solution. Every *pair* of justices produces exactly one handshake, so the total number of handshakes is the number of unordered pairs that can be chosen from nine people, namely $\binom{9}{2}$:

$$\binom{9}{2} = \frac{9!}{7!\,2!} = \frac{9 \times 8 \times \cancel{7!}}{2! \times \cancel{7!}} = \frac{9 \times 8}{2} = \frac{72}{2} = 36.$$

Answer: 36 handshakes.

Alternative Solution:

The first justice shakes hands with the other 8 justices, giving 8 handshakes. The second justice has already shaken hands with the first justice, so they only have 7

new handshakes. The third justice has 6 new handshakes, and so on so the total number of handshakes is:

$$8 + 7 + 6 + ... + 1.$$

Using the identity $1 + 2 + 3 + \cdots + n = \dfrac{n(n+1)}{2}$ with $n = 8$, we get

$$\frac{8(8+1)}{2} = \frac{8 \cdot 9}{2} = 36.$$

Answer: 36 handshakes.

4. Using the identity $1 + 2 + 3 + \cdots + n = \dfrac{n(n+1)}{2}$.

   (a) What is $1 + 2 + 3 + \cdots + 520$?

   Solution. Apply the identity with $n = 520$:

   $$1 + 2 + 3 + \cdots + 520 = \frac{520\,(520+1)}{2} = \frac{520 \times 521}{2} = \frac{270{,}920}{2} = 135{,}460.$$

   Answer: 135,460.

   (b) True or False: $1 + 2 + 3 + \cdots + n = O(n^2)$.

   Solution. By the identity,

   $$1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \frac{n^2 + n}{2},$$

   in which the n^2 term dominates. Formally, $\frac{n^2+n}{2} \leq n^2$ for every $n \geq 1$, so the constants $c = 1$ and $n_0 = 1$ satisfy the definition of big-Oh.

   Answer: True.

   (c) True or False: $1 + 2 + 3 + \cdots + n^2 = O(n^3)$.

   Clarification. The sum $1 + 2 + 3 + \cdots + n^2$ is meant to be read as the sum of all the integers from 1 up to n^2 (the pattern $1, 2, 3, \ldots$ just indicates that we are adding consecutive integers, and the final term n^2 tells us where the sum stops). The last term of this sum is n^2, so we apply the identity with n^2 in place of n:

   Solution. The last term of this sum is n^2, so we apply the identity with n^2 in place of n:

   $$1 + 2 + \cdots + n^2 = \frac{n^2\left(n^2+1\right)}{2} = \frac{n^4 + n^2}{2}.$$

Here the n^4 term dominates, and since n^4 grows strictly faster than n^3, the sum is *not* $O(n^3)$.

Answer: False. (The sum is $O(n^4)$.)

5. Counting asterisks. Consider first the snippet

```
for i = 1 to n
    for j = 1 to n-1
        print "*"
```

Solution. The inner loop prints $n-1$ asterisks, and the outer loop runs the inner loop n times. The amount of work done by the inner loop does not depend on i, so the counts simply multiply:

$$\#\text{asterisks} \;=\; n \cdot (n-1) \;=\; n(n-1).$$

(a) Two sequential (not nested) loops.

```
for i = 1 to n
    print "*"
for j = n-1 downto 1
    print "*"
```

Solution. These two loops run one after the other rather than being nested, so their counts are *added*:

- The first loop takes the values $i = 1, 2, \ldots, n$ and prints n asterisks.
- The second loop takes the values $j = n-1, n-2, \ldots, 1$ and prints $n-1$ asterisks.

$$\#\text{asterisks} \;=\; n + (n-1) \;=\; 2n-1.$$

(b) Nested loops over half-ranges. (Assume n is even.)

```
for i = 1 to n/2
    for j = n/2+1 to n
        print "*"
```

Solution. Count the iterations of each loop separately:

- The outer loop runs $i = 1$ to $n/2$, so it executes $n/2$ times.
- The inner loop runs $j = n/2+1$ to n, so it executes $n - (n/2+1) + 1 = n/2$ times.

The inner count does not depend on i, so the two multiply:

$$\#\text{asterisks} \;=\; \frac{n}{2}\cdot\frac{n}{2} \;=\; \frac{n^2}{4}.$$

(c) Nested loops whose inner bound depends on i. (Assume n is even.)

```
for i = 1 to n
    for j = 1 to i
        print "*"
```

Solution. Here the inner loop runs exactly i times, and this count *changes* with each outer iteration. We therefore sum rather than multiply:

$$\#\text{asterisks} \;=\; \sum_{i=1}^{n} i \;=\; 1+2+3+\cdots+n \;=\; \frac{n(n+1)}{2}.$$

6. With $f(n) = 2n+1$ and $g(n) = n^2$, which of the given pairs c, n_0 are appropriate for proving $f(n) = O(g(n))$ directly from the definition?

Recall that we must exhibit positive constants c and n_0 such that $f(n) \le c\,g(n)$ for *all* $n \ge n_0$. A candidate pair fails as soon as we can point to a single $n \ge n_0$ at which the inequality is violated.

(a) $c = 1$ and $n_0 = 1$.

Solution. Testing at $n = n_0 = 1$ gives $f(1) = 2(1)+1 = 3$ and $1 \cdot g(1) = 1 \cdot 1^2 = 1$. The definition would require $3 \le 1$, which is false.

Answer: No — the inequality already fails at $n = 1$.

(b) $c = 2$ and $n_0 = 1$.

Solution. Testing at $n = n_0 = 1$ gives $f(1) = 3$ and $2 \cdot g(1) = 2 \cdot 1^2 = 2$. The definition would require $3 \le 2$, which is false.

Answer: No — the inequality already fails at $n = 1$.

(c) $c = 2$ and $n_0 = 5$.

Solution. Testing at $n = n_0 = 5$ gives $f(5) = 2(5)+1 = 11$ and $2 \cdot g(5) = 2 \cdot 5^2 = 50$, and indeed $11 \le 50$. The inequality also continues to hold for every larger n, since for $n \ge 5$

$$2n^2 - (2n+1) \;=\; 2n(n-1) - 1 \;\ge\; 2 \cdot 5 \cdot 4 - 1 \;=\; 39 \;>\; 0.$$

Answer: Yes — these are appropriate values.

(d) $c = 200$ and $n_0 = 1$.

Solution. Testing at $n = n_0 = 1$ gives $f(1) = 3$ and $200 \cdot g(1) = 200 \cdot 1^2 = 200$, and indeed $3 \leq 200$. For every $n \geq 1$ we also have

$$200n^2 \;\geq\; 200n \;=\; 2n + 198n \;\geq\; 2n + 1.$$

Answer: Yes — these are appropriate values.

In summary, the appropriate choices are (c) and (d).

7. Which of the following are true?

(a) $3n^2 + 4 = O(n^2)$.

Solution. The n^2 term dominates the constant. For every $n \geq 1$, $3n^2 + 4 \leq 3n^2 + 4n^2 = 7n^2$, so $c = 7$ and $n_0 = 1$ work.

Answer: True.

(b) $n^2 = O(n^3)$.

Solution. Take $c = 1$ and $n_0 = 1$. For every $n \geq 1$,

$$n^3 - n^2 = n^2(n-1) \;\geq\; 0 \qquad \Longrightarrow \qquad n^2 \leq n^3.$$

(For instance at $n = 2$ we have $n^2 = 4$ and $n^3 = 8$, and $8 \geq 4$.) Both functions are monotonically increasing and the gap only widens as n grows, so such constants exist.

Answer: True.

(c) $\frac{1}{2}n^2 = O(\sqrt{n})$.

Solution. Write $\sqrt{n} = n^{1/2}$ and compare the two functions by their ratio:

$$\frac{\frac{1}{2}n^2}{n^{1/2}} \;=\; \frac{1}{2}\, n^{3/2} \;\longrightarrow\; \infty \qquad \text{as } n \to \infty.$$

Because this ratio grows without bound, no constant c can dominate it; that is, there is no choice of c and n_0 with $c\sqrt{n} \geq \frac{1}{2}n^2$ for all $n \geq n_0$.

Answer: False.

(d) $n! = O(n^n)$.

Solution. Write both products out explicitly. Each has exactly n terms:

$$\begin{aligned} n! &= n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdots 2 \cdot 1 && (n \text{ terms}) \\ n^n &= n \cdot n \cdot n \cdot n \cdots n \cdot n && (n \text{ terms}) \end{aligned}$$

Comparing the two products term by term, every factor of n^n equals n, which is greater than or equal to the corresponding factor of $n!$. Hence $n! \leq n^n$ for all $n \geq 1$, and $c = 1$, $n_0 = 1$ work.

Answer: True.

(e) $\log n! = O(n \log n)$.

Solution. Apply the same term-by-term idea to the logarithm, which turns the product into a sum:

$$\begin{aligned} \log n! &= \log \big(n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1\big) \\ &= \log n + \log(n-1) + \log(n-2) + \cdots + \log 2 + \log 1 && (n \text{ terms}) \\ n \log n &= \log n + \log n + \log n + \cdots + \log n + \log n && (n \text{ terms}) \end{aligned}$$

Each term on the upper line is at most the corresponding term $\log n$ on the lower line, so $\log n! \leq n \log n$ for all $n \geq 1$, and $c = 1$, $n_0 = 1$ work.

Alternatively, Stirling's approximation gives $\log(n!) \approx n \log n$ directly.

Answer: True.

8. For each code snippet, determine whether the given property holds at the start of each loop iteration.

Reminder on the convention. "The start of an iteration" means just after i is assigned its new value, before the body executes. This includes the final "dummy" iteration, in which i is assigned a value exceeding the loop's upper bound and the body is therefore never run. A property is a loop invariant only if it holds at every one of these checkpoints, for every input array.

(a)

```
for i = 1 to n
    A[i] = 3
```

Property: all entries of $A[1 : i-1]$ are equal to 3.

Solution. By the time the loop reaches index i, the body has already executed once for each of $1, 2, \ldots, i-1$, and each such execution assigned the value 3 to that position. Every entry up to but not including the current i is therefore equal to 3. At $i = 1$ the range $A[1 : 0]$ is empty, so the property holds vacuously.

Answer: Yes. (Given as the example.)

(b)

```
for i = 1 to n-1
    if A[i] < A[i+1]
        exchange A[i] with A[i+1]
```

Property: the elements in subarray $A[1 : i-1]$ are in increasing order.

Solution. Take the counterexample $A = [\,1, 3, 2\,]$, so $n = 3$ and the loop runs $i = 1$ to 2.

- **Start of $i = 1$:** $A = [\,1, 3, 2\,]$. The range $A[1 : 0]$ is empty, so the property holds vacuously. The body then finds $A[1] = 1 < A[2] = 3$ and swaps, giving $A = [\,3, 1, 2\,]$.
- **Start of $i = 2$:** $A = [\,3, 1, 2\,]$. The range $A[1 : 1] = [\,3\,]$ consists of a single element and is therefore trivially in increasing order, so the property holds. The body then finds $A[2] = 1 < A[3] = 2$ and swaps, giving $A = [\,3, 2, 1\,]$.
- **Start of $i = 3$ (the dummy iteration):** $A = [\,3, 2, 1\,]$. Now $A[1 : 2] = [\,3, 2\,]$, and since $3 > 2$ this is not in increasing order. *The property fails.*

Because the property is violated at the final checkpoint, it is not an invariant.

Answer: No — counterexample $A = [\,1, 3, 2\,]$, which the loop turns into $[\,3, 2, 1\,]$.

Remark. This loop makes a single left-to-right pass and swaps whenever $A[i] < A[i+1]$, so it pushes *larger* elements to the *left*. It therefore tends to leave the front of the array in decreasing order, which is the opposite of what the property claims.

(c)

```
for i = 1 to n-1
    if A[i] < A[i+1]
        exchange A[i] with A[i+1]
```

Property: the element in position $A[i-1]$ (if it exists) is greater than or equal to all the elements in $A[1 : i-2]$.

Solution. Take the counterexample $A = [\,3, 1, 2\,]$, so $n = 3$ and the loop runs $i = 1$ to 2.

- **Start of $i = 1$:** $A = [\,3, 1, 2\,]$. Position $A[0]$ does not exist, so the property holds vacuously. The body checks whether $A[1] = 3 < A[2] = 1$; it is not, so no swap occurs and $A = [\,3, 1, 2\,]$.
- **Start of $i = 2$:** $A = [\,3, 1, 2\,]$. Here $A[i-1] = A[1] = 3$, and the range $A[1:0]$ is empty, so there is nothing to compare against and the property holds vacuously. The body then finds $A[2] = 1 < A[3] = 2$ and swaps, giving $A = [\,3, 2, 1\,]$.
- **Start of $i = 3$ (the dummy iteration):** $A = [\,3, 2, 1\,]$. Now $A[i-1] = A[2] = 2$ and $A[1:1] = [\,3\,]$, so the property would require $2 \geq 3$, which is false. *The property fails.*

Because the property is violated at the final checkpoint, it is not an invariant.

Answer: No — counterexample $A = [\,3, 1, 2\,]$; at $i = 3$ we would need $2 \geq 3$.

9. Insertion-Sort, as given in the textbook:

```
INSERTION-SORT(A, n)
1   for i = 2 to n
2       key = A[i]
3       // Insert A[i] into the sorted subarray A[1 : i-1].
4       j = i - 1
5       while j > 0 and A[j] > key
6           A[j+1] = A[j]
7           j = j - 1
8       A[j+1] = key
```

(a) Run the algorithm by hand on $A = [\,12, 8, 7, 3\,]$, showing the array at the start of each iteration of the outer loop, including the final iteration where $i = n+1$.

Solution. Below, the vertical bar separates the sorted prefix $A[1 : i-1]$ from the part of the array not yet processed.

- **Start of $i = 2$:** $A = [\,12 \mid 8,\ 7,\ 3\,]$. Here $key = A[2] = 8$. Since $12 > 8$, we shift 12 to the right and insert 8 in front of it.
- **Start of $i = 3$:** $A = [\,8,\ 12 \mid 7,\ 3\,]$. Here $key = A[3] = 7$. Since $12 > 7$ we shift 12 right, and since $8 > 7$ we shift 8 right, then place 7 in front.
- **Start of $i = 4$:** $A = [\,7,\ 8,\ 12 \mid 3\,]$. Here $key = A[4] = 3$. Since $12 > 3$, $8 > 3$ and $7 > 3$, all three elements shift right and 3 is placed in front.
- **Start of $i = 5$** (the final "dummy" iteration, $i = n + 1$)**:** $A = [\,3,\ 7,\ 8,\ 12\,]$,

which is fully sorted.

The four required snapshots are therefore

$$\begin{aligned} i = 2: &\quad [\,12,\ 8,\ 7,\ 3\,] \\ i = 3: &\quad [\,8,\ 12,\ 7,\ 3\,] \\ i = 4: &\quad [\,7,\ 8,\ 12,\ 3\,] \\ i = 5: &\quad [\,3,\ 7,\ 8,\ 12\,] \end{aligned}$$

(b) For the same input, how many times is the comparison $A[j] > key$ in Line 5 executed?

Solution. We count the executions pass by pass. Note that when the while-loop ends because j has reached 0, the test $j > 0$ fails first, and by short-circuiting the comparison $A[j] > key$ is not evaluated on that final pass through the test. In each pass below, therefore, the comparison is executed exactly once per shift.

Iteration	Comparisons	Detail
$i = 2$	1	$12 > 8$
$i = 3$	2	$12 > 7, \quad 8 > 7$
$i = 4$	3	$12 > 3, \quad 8 > 3, \quad 7 > 3$
Total	6	

$$1 + 2 + 3 = 6.$$

Answer: 6 comparisons.

(c) More generally, if the input array has n elements in descending order, how many comparisons are performed?

Solution. Notice the pattern from part (b). When the array is in descending order, the *key* chosen at iteration i is smaller than *every* element before it, so the while-loop never stops early: it compares *key* against all $i - 1$ elements to its left, shifting each one, and halts only when j reaches 0. Iteration i therefore performs exactly $i - 1$ comparisons.

Summing over all iterations of the outer loop, $i = 2$ to n:

$$\sum_{i=2}^{n}(i-1) \;=\; \sum_{k=1}^{n-1} k \;=\; \frac{(n-1)\,n}{2} \;=\; \frac{n(n-1)}{2}.$$

As a consistency check against part (b), setting $n = 4$ gives $\frac{4 \cdot 3}{2} = 6$, which agrees with the six comparisons counted there.

Answer: $\dfrac{n(n-1)}{2}$ comparisons. This is the worst case for Insertion-Sort. Note that $\frac{n(n-1)}{2} = \binom{n}{2}$ is exactly the number of pairs of elements in the array, so in the worst case every single pair of elements ends up being compared to each other.

Clarification on formula for further understanding:

$\binom{n}{2}$ vs. the usual sum

$$\sum_{k=1}^{n} k \;=\; 1 + 2 + \cdots + n \;=\; \frac{n(n+1)}{2}$$

$$\sum_{k=1}^{n-1} k \;=\; 1 + 2 + \cdots + (n-1) \;=\; \frac{n(n-1)}{2} \;=\; \binom{n}{2}$$

The first formula is the sum of numbers from 1 to n (or whatever value is on top of the summation). The second sum just has one fewer term (stops at $n-1$, not n) — that's the only difference: $+1 \to -1$.